

Example Lab Report

Adrien Bonansea, Tai Chung, Samuel Jediah Gultom

October 2024

1 Abstract

In this report we investigate the read noise, gain and dark current of a commercial CCD, the Point Grey Chameleon3. We do this by looking at how the Poisson distribution and Gaussian distribution can predict the behavior of photons when having both large and small data sets. We then test various aspects of the such as the the saturation, intensities for exposure time and apertures, and the measurements of noise and dark current. We then explore potential reasons why are noise and dark noise is much larger than anticipated and compared to most CCDs.

2 Introduction

In many parts of astronomy various types of sensors are needed to record optical, infrared, and electromagnetic waves from many far away sources of light. Usually this is done through the photoelectric effect, leading to us determining the wavelengths of photons through various band gaps of energy. In this paper, we will explore the capabilities of a specific kind of device which detects photons, the Charge-Coupled Devices, which will be referred to as CCDs onwards. A standard CCD is built with any semiconductors, know as photosensitive capacitors or pixels. When one of these pixels is hit by an incoming photon it creates an electron-hole pair which is then repelled due to the positive potential the ccd gives off. These electrons are then captured to the next potential well, moving them pixel by pixel to the place where they will be read.(Peterson) This allows astronomers to detect both the energy levels and locations of the incoming photons with signal amplification.

The CCD are also used in astronomy due to their relatively high accuracy in collecting data. However, the CCDs are not perfect, for example their quantum efficiency quantum efficiency is not 100% but high relative to other detectors. quantum efficiency(QE) refers to the to the CCDs ability to convert an incoming photon to an electron that we can digitizes once detected. Most astronomical CCDs will have around an 80% QE. Modern CCDs also have very high charge transfer efficiency, electrons transferred vs electrons produced, estimated to be around 0.99999. A factor that can have quite a large effect on CCDs is dark current. This phenomenon is caused by the device increasing in temperature and consequently imitating the photoelectric effect and move the electrons beyond the Gap band. For this reason, the CCDs are typically used when cool to reduce this effect by a maximum amount. Electron noise can produce a similar effect to dark current in the sense that there is a read out of electrons when there is no light or photons that or shot at the CCD. Finally, the last possible issue with the CCD i will talk about is the conversion gain and full well. At low gain, the CCD is very sensitive to being saturated in some reasons or high levels of noise, while high gain reduces range of the more sensitive signals.

In other cases, astronomers may use a Cmos which works similarly and has the same issues as the CCD. The main difference is how they process electrons. The Cmos will convert the photon, and the charge to voltage at the cell while the CCD will only convert the photon at the cell and the charge at the a final cell which has the read out noise generated.

In this paper, we will go through multiple tests to get data and test various functions of the CCD and its difficulties. For instance, we will take multiple images of a dark and light region to be able to find the readout noise and camera gain and find out if they are skewed compared to what we expect.

3 Data and Observation

3.1 Distributions

The first section of the data and observation will look at a few various ways of how we look at data in CCDs in terms of statistics and distributions. The first important distribution method we will look at is the Poisson distribution, which is defined as a discrete probability distribution which takes a given number of independent events that occur in a set amount of time. It allows us to find the mean and variance of these random events and the probability that they will occur. This is especially important in astronomy due to many astronomers have to deal with sensitive data, such as low signal values from various bodies. The Poisson distribution allows an interpretation of the data measured, and calculation of the probabilities relating to them and their uncertainties. The Poisson equation that we typically use is define by Equation (1).

The Gaussian distribution, commonly referred to as the normal distribution, is a widely used statistical method known for its symmetry around the mean. Its shape is defined by the mean and standard deviation of the dataset, capturing the likelihood of data points falling within certain intervals. The formula for the Gaussian distribution, shown in Equation (2), is often applied to predict values in large datasets, creating smoother and more stable graphs. In other words, the Poisson distribution is often more accurate in small data sets due to it taking into account large deviations, while we can observe that as the data set gets larger, the Gaussian and Poisson distributions converge into a similar shape

$$(1) : \frac{\mu^x}{x!} e^{-\mu} \qquad (2) : f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Below there are examples of taking a small and large data set and showing how the Poisson and Gaussian distributions interpret these sets. Code can be scene in section 7.1.

We will first start by looking at the small data set

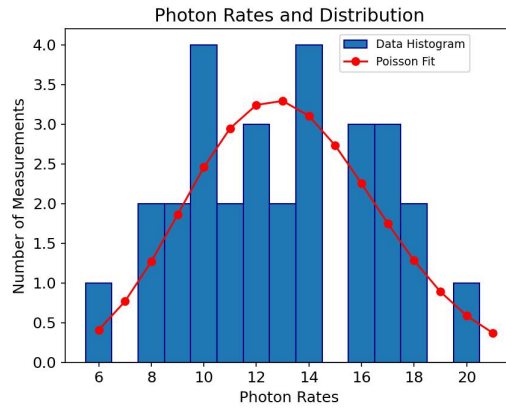


Figure 1: Histogram showing photon rate and Poisson distribution for small data set

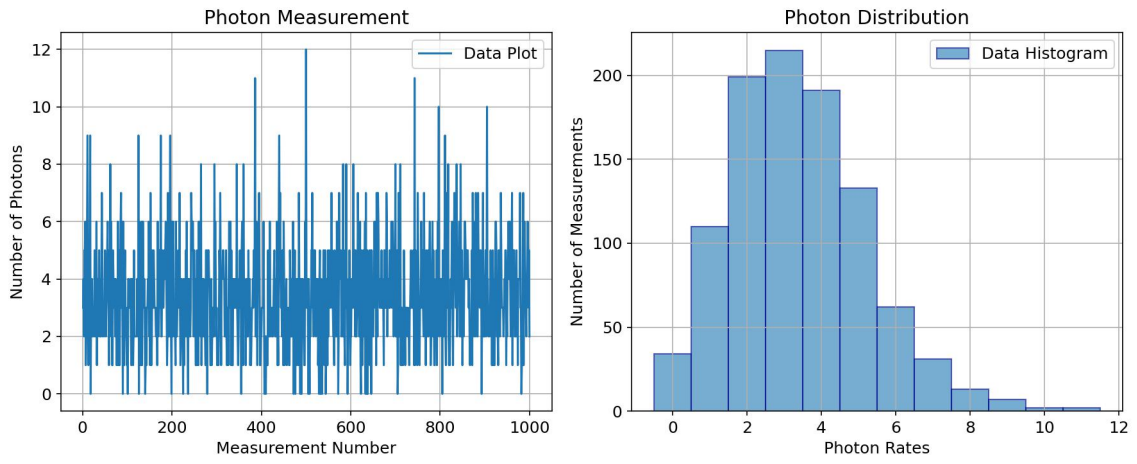


Figure 2: Photon measurements and the distribution associated for small data set

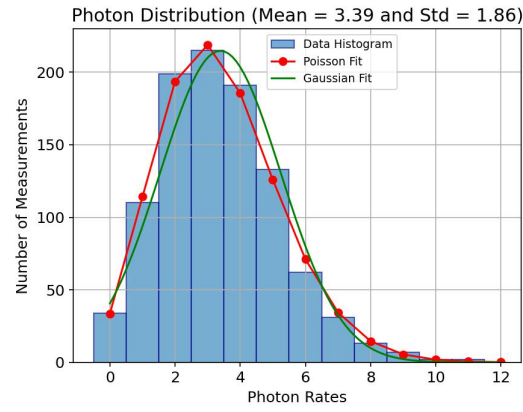


Figure 3: Histogram of photon distribution with both Poisson and Gaussian Distribution for small data set

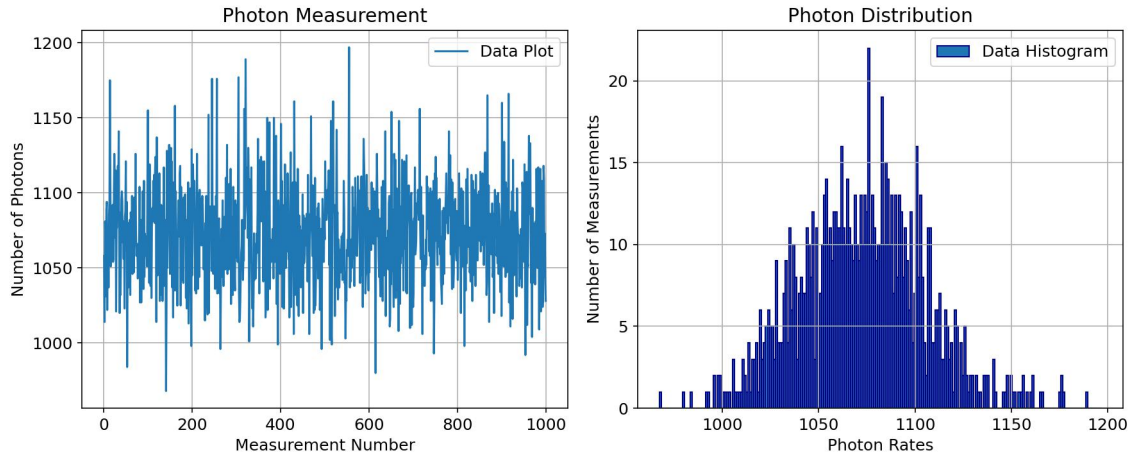


Figure 4: Photon measurements and the distribution associated for large data set

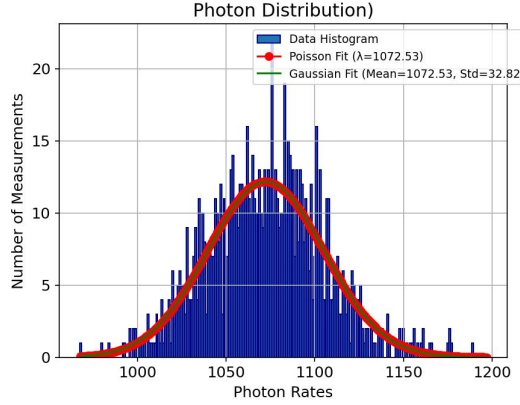


Figure 5: Histogram of photon distribution with both Poisson and Gaussian Distribution for large data set

As we can see in figure 5, the large data set shows a nearly Identical Poisson and Gaussian distribution. Due to the central limit theorem, we can argue that they will give us the same or very similar means which allows us to use a combination of both distribution properties. Code can be scene in section 7.2.

3.2 Modelling

In our analysis of read noise and dark current, we will use the least square method to make predictions of what the noise and the dark current of our data set are. The formula we use for this is:

$$S(\theta) = \sum_{i=1}^n (y_i - f(x_i, \theta))^2$$

3.3 Experimental set up

We use the Flycatch program to manipulate the various aspects of the CCD such as the shutter value and automatic change functions. I will guide how we used the program for data in each of the sections below.

3.3.1 Saturation

We first turn on all automatic options in the control panel of the CCD. We then look at how the histogram shifts when the shutter value is adjusted between 0.046 ms and 1s to find the maximum intensity possible.

3.3.2 Intensity and Exposure Time

To measure intensity and exposure time, we cover the lens with a black cloth to let no or nearly no We then do a similar method to what we did with the saturation part, we vary the shutter value between 0.046 ms to 1 second while observing the histogram and finding the trend when increasing exposure time

3.3.3 Intensity and Aperture

We first take the CCD and point it at a Uniform target, in our case, we used the CCD white box. Ensure that the lens is out of focus to minimize any structures in the image. Adjust the aperture to F1.4, set the gain to a max of 24dB and activate the automatic settings for brightness, exposure, and shutter. With this set up, we can observe the histogram plot and confirm that it stabilizes between 180 and 220 ADU.

Once stabilized, We turn off auto exposure in the control panel and adjust the aperture to F-stop and observe the histogram. We then test the histogram at the largest (F/1.4) and smallest settings (F/8). Finally we repeat this process while pointing the CCD at a non uniform source, somewhere containing a gradient between dark and bright areas.

3.3.4 Measuring Read Noise

In this section we will need to utilize the camera and the computer's terminal. We first cover the CCD with a black cloth as we did in section 3.3.2. After making sure it is well covered, we close the program, open the terminal and utilize the `acquireImages.py` Python function to capture a multitude of images that we will later use to remove some skewed values.

We then perform the same method but with the CCD pointing at an image with a light gradient similar to section 3.3.3. We can then transfer these files to our personal computers and into a jupyter notebook (or any other area for data manipulation) to find the camera gain and noise.

3.3.5 Measuring Dark Current

For the final set up, we put the lens cap on the the CCD, disable all automatic settings and set the brightness to about 2/3 of the automatic values. We then set the gain and frame rate to the maximum. We then choose various shutter values, in our case 1, 5, 10, 15, 20, 25, and 30 and wait 10s after each shutter change before recording the mean intensity at each shutter.

4 Reduction and Results

4.1 Saturation

As we can see from Fig ? below, we observe a range between 0 and 255. This is due to the maximum intensity of a camera being $2^{(number\ of\ bits)}$. As our CCD is an 8 bit camera, we have $2^8 = 256$ giving us this range and so a maximum intensity of 255. We also notice that the mean is around 212 (I do not put the exact value on the figure due to the mean constantly fluctuating between 211 and 213 and happened to be equal to 211.77 at the time of the image taken).

4.2 Intensity and Exposure Time

This part of the experiment went as expected. We first see a uniform black image but as we increase the exposure time the image increases in brightness. Additionally, the increase in exposure time also increased both the mean value of the histogram and how dynamic the image was.

4.3 Intensity and Aperture

As we increase aperture size, we notice that the brightness of the image increases making the intensity of the histogram. Additionally, we see the standard deviation of the histogram decrease. This means

that the distribution of the intensities has less variance throughout the data set. We can see these trends in figure 6 and 7 below where F1.4 is shifted to the right and has a higher peak than F8, signalling higher intestines with less variance. F1.4 has a mean of about 200 ADU and F8 has a mean of about 190 ADU.

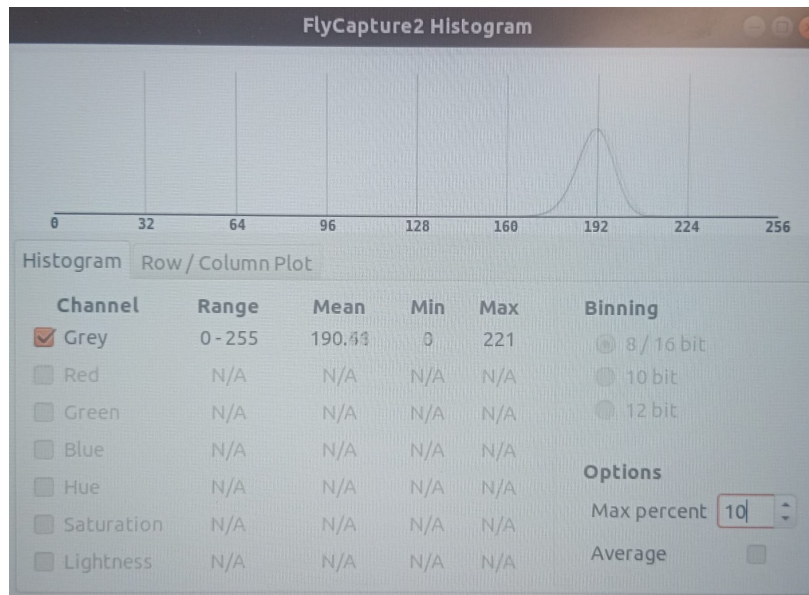


Figure 6: Image of aperture being equal to F8

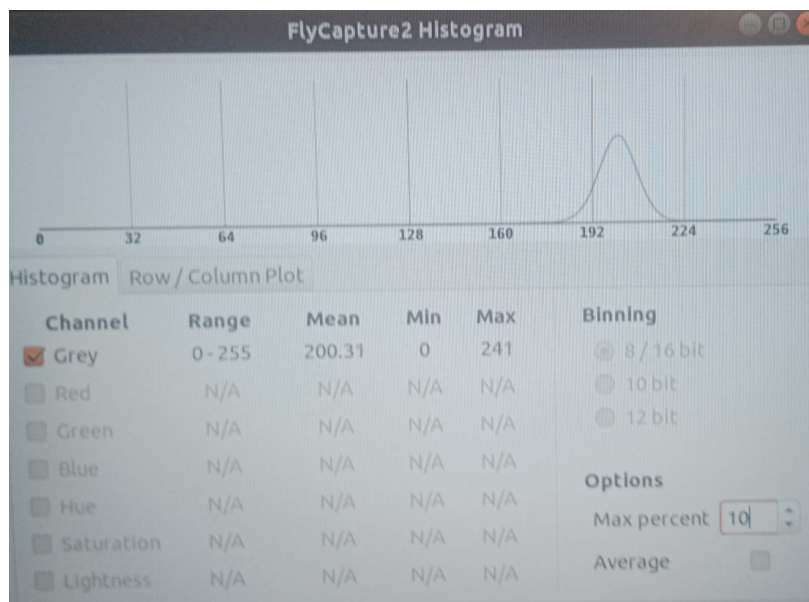


Figure 7: Image of aperture being equal to F1.4

4.4 Measuring Read out Noise

This section is where we study the noise and gain of the CCD. It is also the first section where we have had to do a meaningful reduction of data due to over-saturation of the camera leading to skewed data. after taking the 25 dark and light images, and manipulating the. we use the mean light of the dark images and remove them from the light images to remove any extra light from what we are recording. We can then find the mean light per pixel and variance. To find the noise, we need to graph the data and to find a fit. We take a sample of 1000 points and graph these on a mean by variance graph which is shown in figure 8. We notice that there are many points clustered around the 255 point with nearly 0 variance. These points occur when the CCD is maximized and cause our linear regression to be skewed. Taking into account these points would also cause our CCD gain to be negative and camera noise to be abnormally large (around 10^7).

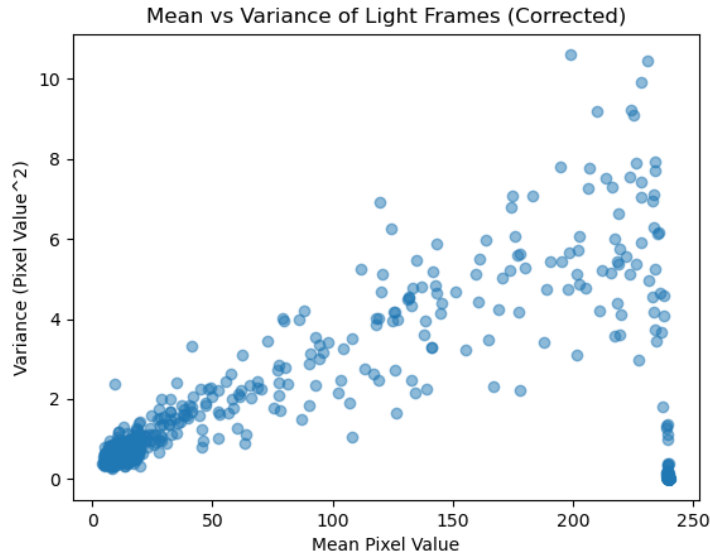


Figure 8: Mean vs Variance of light frames before reduction of values

We then perform a reduction of data, only taking into account points that have a mean pixel value less than 225. We can re run the Jupyter file and find a more normal plot, Figure 9. We can then use the same functions and perform a linear regression to find the gain and the noise of the CCD. Our gain and noise come out to be 35.73 electrons/ ADU and 21.41 electrons respectively.

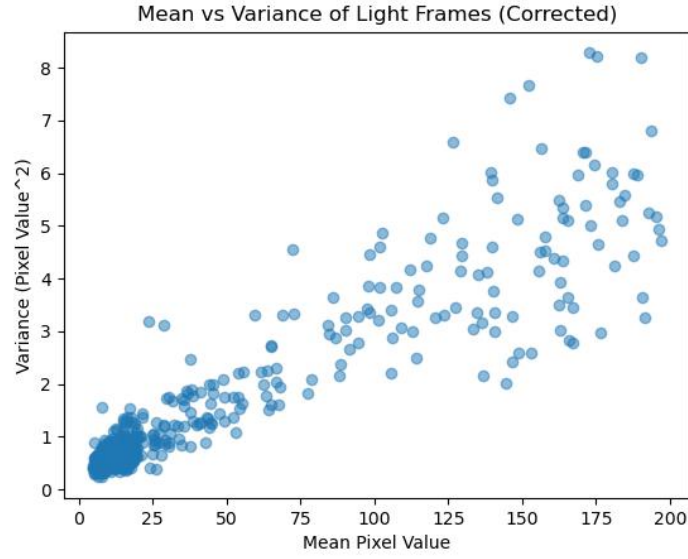


Figure 9: Mean vs Variance of light frames after reduction of values

To increase accuracy of the this data, we can take larger sample values such as 10000 points but for the purpose of this experiment 1000 is enough. Code can be scene in section 7.3.

4.5 Measuring Dark Current

To find the dark value of we first chose various exposure times/ shutter values and found the the mean intensity at that values (Data can be seen in Table 1). We then preform a linear regression

Shutter [ms]	Mean Intensity [ADU/pixel]
1	15.20
5	15.27
10	15.35
15	15.42
20	15.51
25	15.60
30	15.69

Table 1: The mean intensity value for different exposure time

on the data to find the slope of the points given. (Figure 10 and code) By calculating the slope of the red line, we find the dark current present in the CCD by using this value, which is calculated to be 16.7 ADU/ pixel/ second.

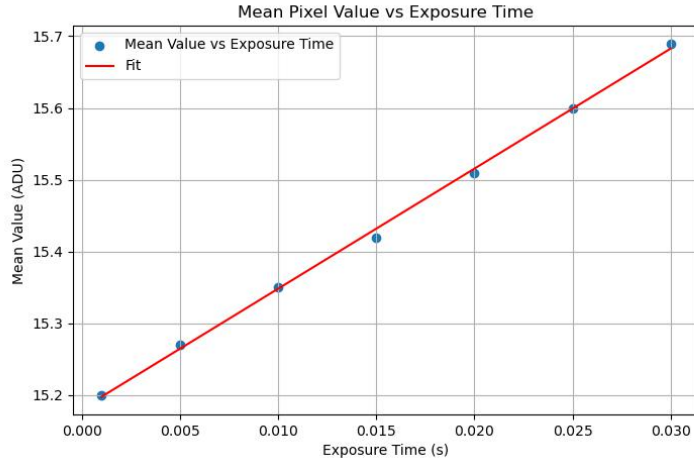


Figure 10: Dark current: men pixel value vs exposure time fit

To see if this linear fit is accurate, we graph the residuals (Figure 11). Through this figure, we see that the largest difference between the fit and a point is less than 0.0125 which can be considered quite large when considering some difference between points are equal to 0.07. This means that this fit is good but could be improved through a by testing various other fits, such as a quadratic fit.

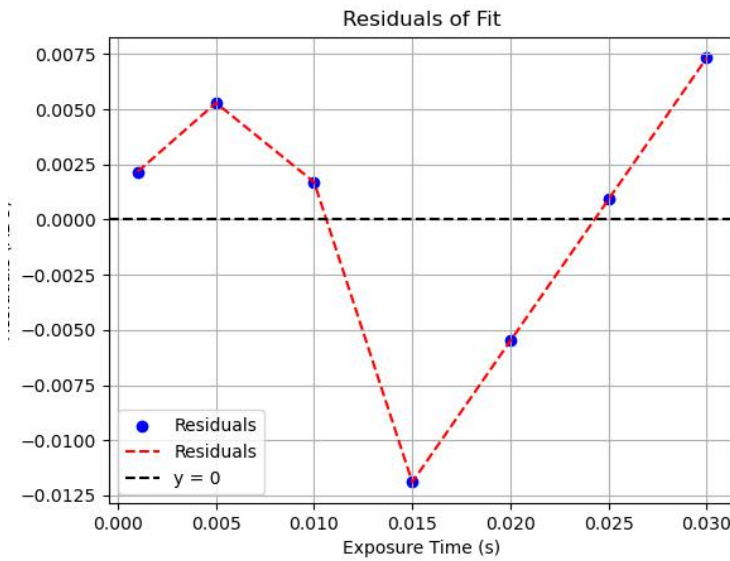


Figure 11: Residuals

TO find the the Dark Current of the CCD in electrons per pixels per second, we have to convert

the multiple the dark current by the gain found in 4.2.4.

$$\begin{aligned}\text{Dark Current [electrons/pixel/second]} &= \text{Dark Current [ADU/pixel/second]} \times \text{CCD Gain} \\ &= 16.7 \times 35.73\end{aligned}$$

$$\text{Dark Current} = 596.8 \text{ electrons/pixel/second}$$

This gives us a Dark current of 597 electrons/pixels/seconds Code can be scene in section 7.4.

5 Discussion and conclusion

in this paper we have studied and tested various aspects of how a CCD works using the Point Grey Chameleon3 model. After reviewing various distribution techniques, we went on to look at the cameras performance under various extremes. We first looked at the camera when it was maximally and minimal saturated by pointing it at a light and putting it in a dark cloth. With this we were able to confirm that the CCD was indeed an 8 bit camera having a range of 0 to 255 ADU and seeing that the increase in exposure will increase the intensity and consequently number of electrons shifted. We then went on to take the light and dark images to get the gain and the noise of the CCD, finding that both of these values were incredibly large. We also find a very large dark current of around 597 electrons/pixel/second. When looking at the dark current and noise of the average CCD we see that our values are extremely high compared to them. There are a multitude of reasons this could be the case.

As explained in the introduction, the dark current is lowered when the CCD is at low temperature. During our practical, the room temperature was already quite a bit higher than what we want the CCD temperature to be, around 26C. Additionally, our CCD was regularly over heating, causing us to have to repeat experiments due to crashing or over-saturating. An abnormally high temperature was caused by the absence of a cooling system in the CCD and would explain why our Dark Current is extremely high, especially as the CCD felt as if it was burning up when touching it. High Dark current leads to more electrons simulating the photoelectric effect which makes us get more recorded gain.

Another reason why our noise was higher than expected could be the sampling. Although unlikely, the data could be shifted due to the sampling of the data leading to smaller or larger linear fits. This can be solved by taking larger samples of data. Finally, our data lacks uncertainties in the fit, this causes us to not know the possible range of the linear fits and so camera noise. This is amplified with by our data being skewed due to the lack of cooling system making the camera overheat.

In the future, having a CCD with a cooling system or finding a way to keep temperature at the optimal value for the CCD to collect data would be great improvements to find more accurate data on the noise of the CCD.

6 References

Peterson, C. (2001). How it works: the charged-coupled device, or CCD. *Journal of young investigators*, 3(1).

7 Appendix

7.1 Code for Small.txt file

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson
from scipy.stats import norm
plt.rcParams['figure.dpi'] = 150 # for crisper figures
plt.rcParams['font.size'] = '12'

# Data
data = [13, 17, 18, 14, 11, 8, 21, 18, 9, 12, 9, 17, 8, 20, 14, 14, 6, 10, 10, 14, 16, 17, 16, 13,

plt.hist(data, bins=np.arange(min(data), max(data) + 1) - 0.5, edgecolor='darkblue', label='Data H

# Fit and plot Poisson distribution
mean = np.mean(data)
x = np.arange(min(data), max(data) + 1)
poisson_pmf = poisson.pmf(x, mean) * len(data)

# Overlay the Poisson distribution
plt.plot(x, poisson_pmf, 'red', marker='o', label='Poisson Fit')

# Add labels and legend
plt.xlabel('Photon Rates')
plt.ylabel('Number of Measurements')
plt.title('Photon Rates and Distribution')
plt.legend(loc='upper left', bbox_to_anchor=(0.6, 1), fontsize=9, frameon=True, borderpad=0.4, lab

# Show plot
plt.savefig("Small_data_photonrates_dist.jpg")
plt.show()

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson
from scipy.stats import norm
plt.rcParams['figure.dpi'] = 150 # for crisper figures
plt.rcParams['font.size'] = '12'

# Data
data = [13, 17, 18, 14, 11, 8, 21, 18, 9, 12, 9, 17, 8, 20, 14, 14, 6, 10, 10, 14, 16, 17, 16, 13,

plt.hist(data, bins=np.arange(min(data), max(data) + 1) - 0.5, edgecolor='darkblue', label='Data H

# Fit and plot Poisson distribution
mean = np.mean(data)
```

```

x = np.arange(min(data), max(data) + 1)
poisson_pmf = poisson.pmf(x, mean) * len(data)

# Overlay the Poisson distribution
plt.plot(x, poisson_pmf, 'red', marker='o', label='Poisson Fit')

plt.xlabel('Photon Rates')
plt.ylabel('Number of Measurements')
plt.title('Photon Rates and Distribution')
plt.legend(loc='upper left', bbox_to_anchor=(0.6, 1), fontsize=9, frameon=True, borderpad=0.4, label)
plt.savefig("Small_data_photonrates_dist.jpg")
plt.show

import numpy as np
import matplotlib.pyplot as plt
import statistics
from scipy.stats import poisson
plt.rcParams['figure.dpi'] = 150
plt.rcParams['font.size'] = '12'

# Data
data = np.loadtxt('Small.txt')

# Examine the mean and standard deviation of the data
mean_data = np.mean(data)
stdev_data = statistics.pstdev(data)

# Define x values for the Poisson PMF and Gaussian PDF
pois = np.arange(min(data), max(data) + 1)
gaus = np.linspace(min(data), max(data), 100)

# Plotting histogram
plt.hist(data, bins=np.arange(min(data), max(data) + 1) - 0.5, alpha=0.6, edgecolor='darkblue', label)

# Calculate the Poisson PMF and Gaussian PDF
poisson_pmf = poisson.pmf(pois, mean_data) * len(data)
plt.plot(xp, poisson_pmf, 'red', marker='o', label='Poisson Fit')

gaussian_pdf = norm.pdf(gaus, mean_data, stdev_data) * len(data)
plt.plot(xg, gaussian_pdf, 'green', label='Gaussian Fit')

plt.title('Photon Distribution (Mean = {:.3} and Std = {:.3})'.format(mean_data, stdev_data))
plt.xlabel('Photon Rates')
plt.ylabel('Number of Measurements')
plt.grid(True)
plt.legend(loc='upper left', bbox_to_anchor=(0.42, 1), fontsize=9, frameon=True, borderpad=0.4, label)
plt.savefig("Large_data_photonrates_dist.jpg")
plt.show()

```

7.2 Code for Large.txt file

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 150
plt.rcParams['font.size'] = '12'

# Data
data = np.loadtxt('Large.txt')

# Examine the mean and stdev of the data
mean_data = np.mean(data)
stdev_data = statistics.pstdev(data)

# Create a figure with 1 row and 2 columns of subplots
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))

# Plotting in the first subplot (left)
x = np.arange(1, len(data) + 1)
axs[0].plot(x, data, label='Data Plot')
axs[0].set_title('Photon Measurement')
axs[0].set_xlabel('Measurement Number')
axs[0].set_ylabel('Number of Photons')
axs[0].grid(True)
axs[0].legend()

# Plotting in the second subplot (right)
axs[1].hist(data, bins=np.arange(min(data), max(data) + 1) - 0.5, edgecolor='darkblue', label='Data Distribution')
axs[1].set_title('Photon Distribution')
axs[1].set_xlabel('Photon Rates')
axs[1].set_ylabel('Number of Measurements')
axs[1].grid(True)
axs[1].legend()

plt.tight_layout()
plt.savefig("Large_data_photonmeasur_dist.jpg")
plt.show()

import numpy as np
import matplotlib.pyplot as plt
import statistics
from scipy.stats import poisson
plt.rcParams['figure.dpi'] = 150
plt.rcParams['font.size'] = '12'

# Data
data = np.loadtxt('Large.txt')
```

```

# Examine the mean and standard deviation of the data
mean_data = np.mean(data)
stdev_data = statistics.pstdev(data)

# Define x values for the Poisson PMF and Gaussian PDF
xp = np.arange(min(data), max(data) + 1)
xg = np.linspace(min(data), max(data), 100)

# Plotting histogram
plt.hist(data, bins=np.arange(min(data), max(data) + 1) - 0.5, edgecolor='darkblue', label='Data H

# Calculate the Poisson PMF and Gaussian PDF
poisson_pmf = poisson.pmf(xp, mean_data) * len(data)
plt.plot(xp, poisson_pmf, 'red', marker='o', label=f'Poisson Fit (Mean={mean_data:.2f})')

gaussian_pdf = norm.pdf(xg, mean_data, stdev_data) * len(data) # Scale by number of samples to ma
plt.plot(xg, gaussian_pdf, 'green', label=f'Gaussian Fit (Mean={mean_data:.2f}, Std={stdev_data:.2

# Plot titles and labels
plt.title('Photon Distribution')
plt.xlabel('Photon Rates')
plt.ylabel('Number of Measurements')
plt.grid(True)
plt.legend(loc='upper left', bbox_to_anchor=(0.42, 1), fontsize=9, frameon=True, borderpad=0.4, la
plt.savefig("Large_data_poison_dist.jpg")
plt.show()

```

7.3 Code for section 4.4

```

import numpy as np
import PIL
from PIL import Image
import matplotlib.pyplot as plt
import glob
from random import sample

#define directories and file name files

path = "/Users/adrienbonansea/Downloads"

datadir = "/Users/adrienbonansea/Downloads"

dark_folder = '4.2.4_new_dark_group_F'

light_folder = '4.2.4_new_light_group_F'

dark_filename = 'dark_group_F.txt'

```

```

light_filename = 'light_group_F.txt'

dark_image_names_glob_pattern = glob.glob('{}/{}/Capture*'.format(datadir, dark_folder))

light_image_names_glob_pattern = glob.glob('{}/{}/Capture*'.format(datadir, light_folder))

#check lengths are correct (expected 25)
print(len(dark_image_names_glob_pattern))
print(len(light_image_names_glob_pattern))

dark_images_list = []

for name in dark_image_names_glob_pattern:

    dark_images_list.append(np.array(Image.open(name)))

light_images_list = []

for name in light_image_names_glob_pattern:

    light_images_list.append(np.array(Image.open(name)))

dark_images = np.array(dark_images_list)

light_images = np.array(light_images_list)

#Verify shapes
print('shape of one dark image: {}'.format(np.shape(dark_images_list[0])))

print('shape of dark image array: {}'.format(np.shape(dark_images)))

print('shape of one light image: {}'.format(np.shape(light_images_list[0])))

print('shape of light image array: {}'.format(np.shape(light_images)))

mean_dark_image = np.mean(dark_images, axis = 0)

mean_light_image = np.mean(light_images, axis = 0)

#writing a loop is optional
corrected_light_images = []
for img in light_images:
    corrected_light_images.append(img-mean_dark_image)
correctedL = light_images - mean_dark_image
print(mean_dark_image)

corrected_mean = np.mean(corrected_light_images, axis=0)

```

```

var_light = np.var(corrected_light_images, axis=0)
print(corrected_mean)

mean_flat = corrected_mean.flatten()
var_flat = var_light.flatten()

sample_indices = sample(range(mean_flat.size), 1000)

mean_sample = mean_flat[sample_indices]
#choose max value for initial test (255 or 300 to be safe)
mean_sample_in = mean_sample < 225
mean_sample = mean_sample[mean_sample_in]
var_sample = var_flat[sample_indices]
var_sample = var_sample[mean_sample_in]

plt.scatter(mean_sample, var_sample, alpha=0.5)
plt.title('Mean vs Variance of Light Frames (Corrected)')
plt.xlabel('Mean Pixel Value')
plt.ylabel('Variance (Pixel Value^2)')
plt.savefig("_")
plt.show()

A = np.vstack([mean_sample, np.ones(len(mean_sample))]).T

gain, noise = np.linalg.lstsq(A, var_sample, rcond=None)[0]

print(f"Estimated gain: {1/gain}")
print(f"Estimated noise: {np.sqrt(noise * (1/gain) ** 2)}")

```

7.4 Code for section 4.5

```

import numpy as np
from scipy.stats import linregress
import matplotlib.pyplot as plt

shutters_raw = np.array([1, 5, 10, 15, 20, 25, 30])
mean_values = np.array([15.20, 15.27, 15.35, 15.42, 15.51, 15.60, 15.69])

shutters_in_s = shutters_raw / 1000 #now in s

slope, intercept, r_value, p_value, std_err = linregress(shutters_in_s, mean_values)

line = intercept + slope * shutters_in_s

# Plot the data and the linear fit
plt.figure(figsize=(8, 5))
plt.scatter(shutters_in_s, mean_values, label="Mean Value vs Exposure Time")

```

```

plt.plot(shutters_in_s, line, color='r', label="Fit")
plt.xlabel("Exposure Time (s)")
plt.ylabel("Mean Value (ADU)")
plt.legend()
plt.title("Mean Pixel Value vs Exposure Time")
plt.grid(True)
plt.savefig("4.2.5.jpg")
plt.show()

print(f"The slope of the fit is:{slope} and the intercept is: {intercept}")

residuals = mean_values - line

# Plot residuals
plt.figure()
plt.scatter(shutters_in_s, residuals, color='blue', label="Residuals")
plt.plot(shutters_in_s, residuals, 'red', label="Residuals", linestyle="--")
plt.axhline(y=0, color='black', linestyle='--', label="y = 0")
plt.xlabel("Exposure Time (s)")
plt.ylabel("Residuals (ADU)")
plt.title("Residuals of Fit")
plt.legend()
plt.grid(True)
plt.savefig("4.2.5_res.jpg")

```